SpMV CSR-Based Methods Investigation on GPU

1st Guglielmo Boi
Department of Information Engineering and Computer Science
Università di Trento
Trento, Italy
guglielmo.boi@studenti.unitn.it

***Abstract*—This work investigates three GPU implementations of sparse matrix-vector multiplication (SpMV) based on compressed sparse row (CSR) format: CSR-Vector, CSR-Stream and CSR-Adaptive. These algorithms were evaluated on an NVIDIA A30 GPU using matrices from the SuiteSparse Matrix Collection, and were then compared with the cuSPARSE implementation.**

***Index Terms*—SpMV, CUDA, GPU computing, CSR format, sparse matrices, parallel computing.**

I. Introduction

Sparse matrix-vector multiplication (SpMV) is a key linear algebra algorithm of great importance in computational science [1]. The application of SpMV is extensive and includes engineering computing, graph algorithms, and linear programming, among others [2].

Its formal representation is represented by (I), where A is a sparse matrix of dimension $r \times c$ and x and y are two dense vectors of dimension $c \times 1$ and $r \times 1$, respectively.

$$y = Ax \tag{1}$$

The challenges of SpMV optimization stems from the non-uniform distribution of non-zero (NZ) elements in sparse matrices and its strongly memory-bound nature. In the past years researchers have developed a great amount of formats for storing sparse matrices in a efficient way.

The CSR format optimizes the COO format by compressing the row indices array (rowIdx), while keeping the column indices array (colIdx) and values array (value) unchanged. CSR uses a row pointer array (rowPtr) to store the cumulative number of non-zeroes (NNZ) elements per row.

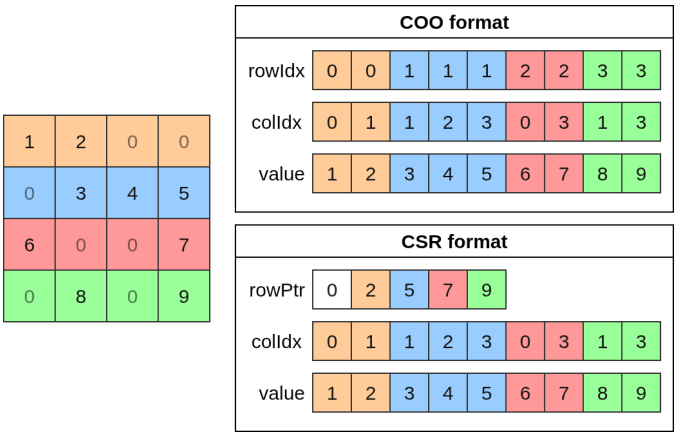

Fig. 1. Diagram showing the COO and CSR formats for sparse matrices.

More recently researches have proposed new storage formats, but unfortunately transforming CSR into these formats has significant runtime and memory overheads [3].

This investigation compares the performance of three main methods for SpMV with the CSR format: CSR-Vector, CSR-Stream and CSR-Adaptive. A CPU implementation is also provided to test the correctness of the algorithms.

II. Methodology

a) CSR-Vector: CSR-Vector assigns one warp to each matrix row [3]. Threads inside the warp collaboratively compute the dot product between the sparse row and the dense vector. Each thread processes a subset of the NZ elements of the row and a warp-level reduction is then performed to accumulate the partial sums.

This strategy improves memory coalescing and reduces thread divergence for rows with a sufficiently large NNZ. However, when rows contain only a few NZ elements, some threads in the warp remain underutilized.

Algorithm 1 CSR-Vector

```
for each row assigned to a warp do
    sum ← 0
    for each NZ handled by the thread do
        sum ← sum + value[j] · x[colIdx[j]]
    end for
    Perform warp-level reduction
    if thread is lane 0 then
        y[row] ← sum
    end if
end for
```

b) CSR-Stream: CSR-Stream assigns one thread to each matrix row [3]. Every thread sequentially processes all the NZ elements belonging to its assigned row and computes the corresponding output value.

This approach is simple and efficient for matrices with short rows, since it minimizes synchronization overhead. Nevertheless, matrices with highly irregular row lengths may lead to load imbalance, because threads assigned to longer rows require significantly more execution time.

Algorithm 2 CSR-Stream

```
for each row assigned to a thread do
    sum ← 0
    for j = rowPtr[row] to rowPtr[row + 1] − 1 do
        sum ← sum + value[j] · x[colIdx[j]]
    end for
    y[row] ← sum
end for
```

c) CSR-Adaptive: CSR-Adaptive combines the advantages of CSR-Vector and CSR-Stream. Rows with a NNZ lower than the number of threads per warp are processed using a thread-based strategy, while rows with larger workloads are assigned to an entire warp.

The selection strategy is based on a threshold on the NNZ elements per row. This adaptive approach aims to reduce the performance penalties caused by irregular sparsity patterns.

d) Validation: To verify the correctness of the GPU implementations, a reference CPU implementation of SpMV using the CSR format was developed. For each matrix, the output vectors produced by the GPU kernels were compared against the CPU results.

The validation algorithm compares each element of the CPU output vector with each element of the GPU output vector, by checking that the absolute error and the relative error are both above a constant threshold `EPSILON`.

III. Dataset

a) SuiteSparse matrices: For the evaluation phase we used 10 matrices that were also employed and reported in the performance evaluation section in [1].

TABLE I
SELECTED SUITESPARSE MATRICES USED IN THE EVALUATION

Matrix	Size	NNZ	Symmetry	Sparsity
ASIC_680ks	682,712	1,693,767	Unsymmetric	Extreme
FullChip	2,987,012	26,621,564	Unsymmetric	Extreme
Ga41As41H72	268,096	18,488,476	Symmetric	Moderate
Rucci1	1,977,885	7,791,168	Unsymmetric	Extreme
Si41Ge41H72	185,639	15,011,265	Symmetric	Moderate
boyd2	466,385	1,502,827	Unsymmetric	Extreme
eu-2005	862,664	19,235,140	Unsymmetric	Slight
ldoor	952,203	42,493,817	Symmetric	Moderate
rajat31	4,690,002	20,316,253	Unsymmetric	Extreme
webbase-1M	1,000,005	3,105,536	Unsymmetric	Extreme

The dataset is selected from the SuiteSparse Matrix Collection in order to cover a wide range of matrix sizes, sparsity patterns, and structural properties. The NNZ ranges from 1M to 47M.

Matrices such as `Ga41As41H72`, `Si41Ge41H72`, and `ldoor` exhibit moderate sparsity structures and stable row lengths. In contrast, matrices such as `webbase-1M`, `eu-2005`, and `FullChip` contain highly irregular row distributions that stress load balancing and memory bandwidth.

b) Random dense vectors: Dense vectors are randomly generated by sampling floating-point values from a uniform distribution within the range of 0 to 1000.

IV. Experimental Setup

The GPU implementations were tested on a NVIDIA A30 GPU with 24 GB of memory. The analysis focuses on kernel execution time, computational throughput, and transferred memory. The SpMV number of floating operations is conventionally measured as $2 \cdot NNZ$. Several runs were performed: minimum, maximum, mean, and standard deviation (SD) values are reported.

The proposed algorithms are compared with the cuSPARSE SpMV implementation (`CUSPARSE_SPMV_ALG_DEFAULT`) as baseline reference.

NVIDIA Nsight Compute (ncu) was used to export and analyze various metrics during execution, such as DRAM and cached transferred memory, throughput, and memory bandwidth.

V. Results

a) Kernel Execution Time: Table II reports the average kernel execution time measured for each matrix and algorithm. The results show a significant variation in performance depending on the sparsity structure of the matrices.

Among the proposed methods, CSR-Stream generally achieved the lowest execution times for several matrices, particularly for `ASIC_680ks`, where it obtained a speedup of approximately $8.6\times$. CSR-Vector also showed competitive performance on matrices with more regular row distributions, achieving a speedup of about $2.7\times$ on the same dataset.

In contrast, the performance of the methods varied considerably on irregular matrices such as `FullChip`, `rajat31`, and `webbase-1M`. In these cases, the proposed kernels were heavily slower than cuSPARSE, because efficiently balancing workloads in highly irregular sparse matrices is difficult.

CSR-Adaptive showed more stable behavior across different datasets by combining thread-based and warp-based execution strategies. However, its execution times were underwhelming.

b) Kernel GFLOPS: Figure 2 shows the computational throughput achieved by each algorithm in terms of GFLOPS.

CSR-Vector achieved the highest throughput among the custom implementations on matrices with regular sparsity patterns. In particular, it reached more than 50 GFLOPS on `Ga41As41H72` and `Si41Ge41H72`, where rows contain a sufficiently large and balanced number of NZ elements.

CSR-Stream obtained more consistent throughput across the dataset, typically achieving between 20 and 30 GFLOPS. Its simpler execution model reduced synchronization overhead and provided stable performance for matrices with shorter rows, such as `FullChip` and `boyd2`.

CSR-Adaptive achieved intermediate performance between CSR-Vector and CSR-Stream for some matrices, while it performed poorly for some others.

The cuSPARSE implementation consistently outperformed the custom kernels on all matrices with the exception

TABLE II
KERNEL EXECUTION TIME (MS) COMPARISON ACROSS SPMV IMPLEMENTATIONS. SPEEDUP IS COMPUTED WITH RESPECT TO CUSPARSE.

Matrix	CSR-Vector					CSR-Stream					CSR-Adaptive				
	Min	Max	Mean	SD	Speedup	Min	Max	Mean	SD	Speedup	Min	Max	Mean	SD	Speedup
ASIC_680ks	0.608	0.697	0.663	0.034	2.725	0.190	0.223	**0.210**	0.011	8.613	57.812	59.104	58.354	50.115	0.378
FullChip	12.430	13.083	12.747	0.207	0.034	2.302	2.330	**2.314**	0.012	0.186	14.425	14.581	14.499	0.056	0.030
Ga41As41H72	0.357	0.366	**0.361**	0.002	0.438	0.640	0.645	0.642	0.001	0.246	0.747	0.756	0.751	0.003	0.211
Rucci1	2.024	2.045	2.035	0.006	0.086	0.679	0.684	**0.682**	0.001	0.257	0.712	0.721	0.717	0.002	0.244
Si41Ge41H72	0.262	0.275	**0.268**	0.003	0.516	0.510	0.515	0.512	0.001	0.270	0.603	0.613	0.608	0.003	0.228
boyd2	0.381	0.390	0.387	0.003	0.127	0.076	0.082	**0.079**	0.002	0.623	0.129	0.134	0.131	0.001	0.373
eu-2005	1.104	1.120	**1.113**	0.005	0.248	1.386	1.398	1.391	0.003	0.199	1.797	1.810	1.802	0.003	0.153
ldoor	1.164	1.178	**1.173**	0.004	0.279	1.712	1.722	1.717	0.002	0.191	2.243	2.263	2.255	0.005	0.145
rajat31	4.779	4.798	4.787	0.005	0.082	1.816	1.821	**1.819**	0.002	0.217	1.909	1.926	1.918	0.004	0.205
webbase-1M	1.032	1.038	1.037	0.002	0.093	0.284	0.289	**0.286**	0.001	0.336	0.334	0.342	0.339	0.002	0.284

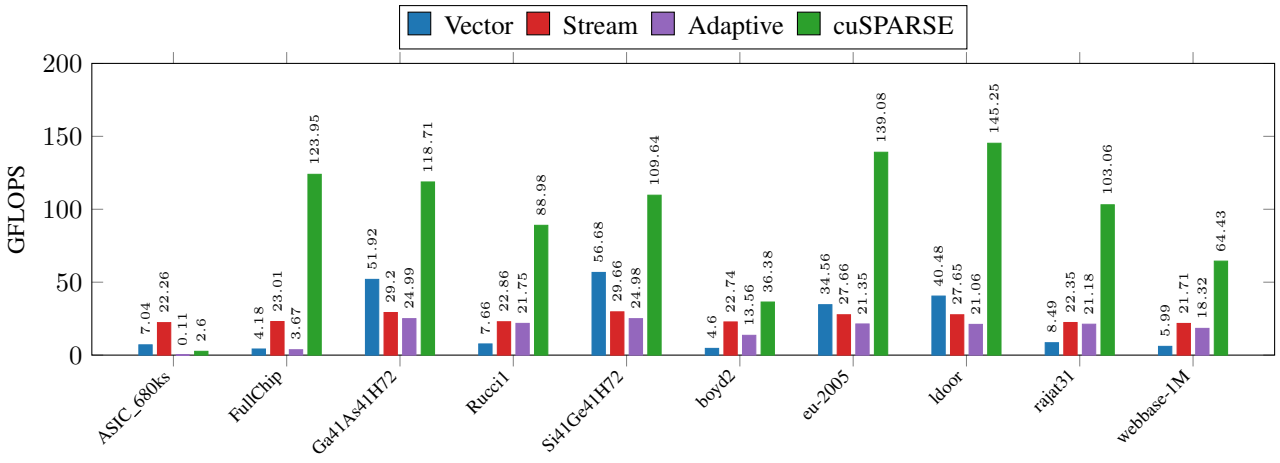

Fig. 2. Kernel GFLOPS comparison across SpMV implementations. FLOPS are measured as $\frac{2 \cdot NNZ}{Kernel\ Time}$.

of `ASIC_680ks`, reaching over 100 GFLOPS on several datasets.

c) Transferred Memory: Figure 3 reports the amount of transferred memory, distinguishing between uncached DRAM traffic and cached memory traffic (L1/L2 cache).

The results confirm that SpMV is strongly memory-bound, with cache traffic often exceeding the amount of direct DRAM accesses. In particular, matrices such as `FullChip`, `ldoor`, `rajat31`, and `webbase-1M` generated very large cache traffic across all implementations.

CSR-Vector generally produced the lowest amount of transferred memory among the proposed kernels, due to the warp-level cooperation which reduces redundant memory accesses.

CSR-Stream exhibited larger memory traffic on several datasets, particularly for matrices with irregular sparsity patterns.

CSR-Adaptive generated the largest total memory traffic on several matrices, because of reduced cache locality introduced by dynamically switching between thread-based and warp-based execution strategies. The transferred memory amount was particularly large for `FullChip`, `ldoor`, `rajat31`, and `webbase-1M`.

VI. DISCUSSION

The results show that performance of CSR-based SpMV is tightly coupled to the structural properties of the input matrices. No single strategy is universally optimal.

CSR-Vector performs best on matrices with regular and sufficiently dense rows such as `Ga41As41H72` and `Si41Ge41H72`, where warp-level cooperation is well utilized and memory coalescing is effective, reaching over 50 GFLOPS. On matrices with highly variable NNZ elements per row, however, many threads remain idle and efficiency degrades significantly.

CSR-Stream's simpler model avoids warp synchronization, achieving consistently competitive throughput between 20 and 30 GFLOPS across the dataset. Its main weakness is load imbalance on matrices with denser matrices.

CSR-Adaptive performs poorly because the adaptive strategy introduces additional overhead and thread divergence. Since different rows may follow different execution paths, GPU warps become less efficient. Overall, the proposed CSR-Adaptive algorithm suffers from the overhead of adapting between thread-based and warp-based execution, which outweighs the potential benefits. It also generated the largest total memory traffic on several matrices due to reduced cache locality.

Figure 4 shows the average Streaming Multiprocessor (SM) utilization and average DRAM bandwidth (BW) utilization.

In several matrices, high bandwidth utilization did not correspond to equally high SM throughput, indicating that

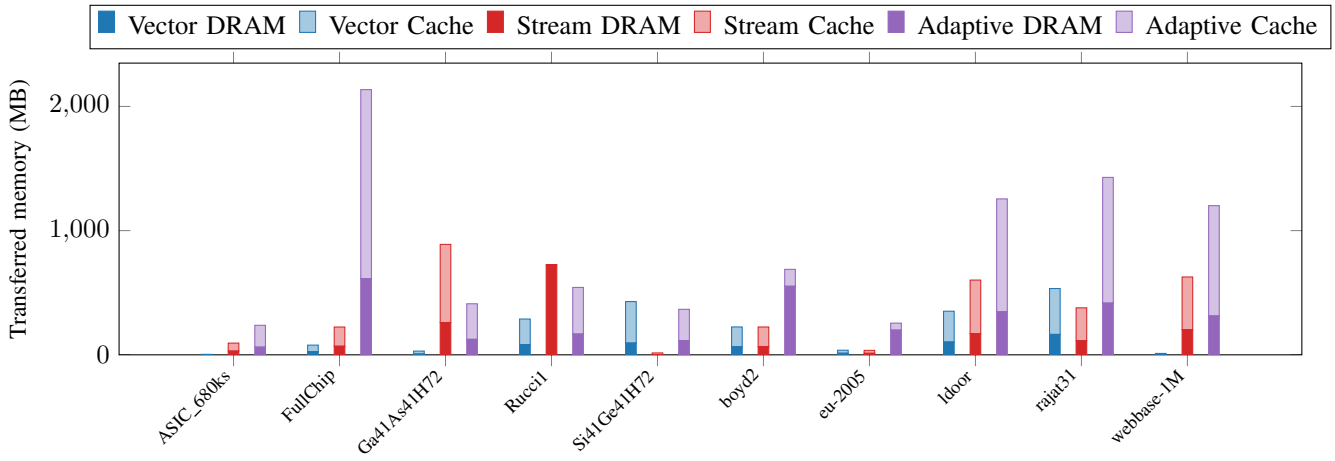

Fig. 3. Transferred memory comparison across SpMV implementations. Both DRAM and L1/L2 caches are reported.

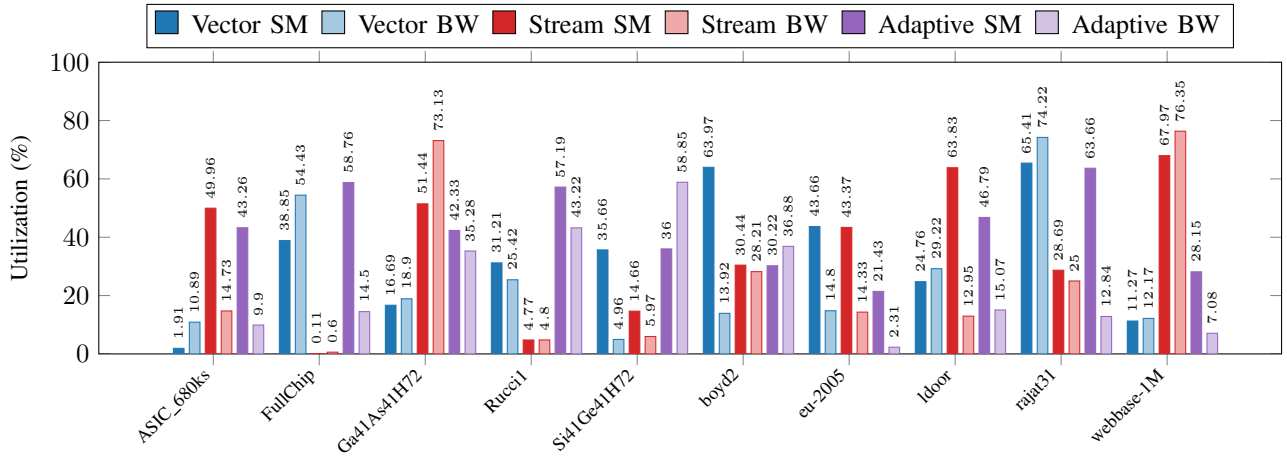

Fig. 4. Average Streaming Multiprocessor compute throughput as a percentage of the GPU's theoretical peak and average DRAM bandwidth utilization as a percentage of the GPU's theoretical peak.

memory accesses remain the primary bottleneck.

CSR-Vector generally obtained higher SM throughput on regular sparsity patterns, while CSR-Stream achieved higher bandwidth utilization on irregular ones. CSR-Adaptive showed inconsistent behavior across the dataset.

VII. Conclusions

This work investigated three CSR-based SpMV implementations on GPU (CSR-Vector, CSR-Stream, and CSR-Adaptive) and compared them against the cuSPARSE baseline across a diverse set of sparse matrices.

The results confirm that SpMV is strongly memory-bound, and that the efficiency of each method depends heavily on the sparsity structure of the input.

CSR-Vector is preferable for structured matrices with regular row distributions, while CSR-Stream provides more stable performance across irregular patterns.

CSR-Adaptive did not achieve competitive performance in its current form due to divergence and memory overhead.

Future work could focus on improving the row classification strategy of CSR-Adaptive and exploring hybrid approaches that better exploit the memory hierarchy of modern GPUs.

References

[1] Chu, G., He, Y., Dong, L., Ding, Z., Chen, D., Bai, H., ... & Hu, C. (2023, August). Efficient algorithm design of optimizing spmv on gpu. In Proceedings of the 32nd international symposium on high-performance parallel and distributed computing (pp. 115-128).

[2] Gao, J., Liu, B., Ji, W., & Huang, H. (2024). A systematic literature survey of sparse matrix-vector multiplication. arXiv preprint arXiv:2404.06047.

[3] Greathouse, J. L., & Daga, M. (2014, November). Efficient sparse matrix-vector multiplication on GPUs using the CSR storage format. In SC'14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (pp. 769-780). IEEE.